Assignment 1: Scholarship Allocation

Course: CS101 Computational Thinking | B.Tech 1st Year

Problem Statement

Design a Python system that evaluates applicants using aggregate marks, subject-wise scores, attendance, and income limits. It must filter eligible students, rank them based on aggregate score, and resolve ties using attendance.

Algorithm

1. Initialize a list of tuples containing applicant data (Name, Math, Sci, Eng, Attendance, Income).
2. Loop through each tuple and unpack the variables.
3. Apply conditional statements to verify minimum requirements.
4. If eligible, calculate the aggregate and append to a new nested list.
5. Sort the nested list using nested loops (Bubble Sort logic).
6. Print the final ranked list.

Python Implementation

```
# (Name, Math, Science, English, Attendance%, Income)
applicants = [
    ("Alice", 85, 90, 88, 95, 40000),
    ("Bob", 70, 65, 80, 70, 50000),
    ("Charlie", 90, 90, 83, 98, 42000),
    ("Dave", 90, 90, 83, 90, 39000) # Tie case
]

eligible = []
for app in applicants:
    name, m, s, e, att, inc = app
    if m >= 75 and s >= 75 and e >= 75 and att >= 80 and inc <= 45000:
        agg = (m + s + e) / 3
        # Store as [Aggregate, Attendance, Name] to easily rank
        eligible.append([agg, att, name])

# Sort descending (Bubble sort approach for syllabus compliance)
n = len(eligible)
for i in range(n):
    for j in range(0, n - i - 1):
        # Tie-breaker logic on index 1 (attendance) if index 0 (agg) is equal
        if eligible[j][0] < eligible[j+1][0] or (eligible[j][0] == eligible[j+1][0]
and eligible[j][1] < eligible[j+1][1]):
            # Swap elements
            temp = eligible[j]
            eligible[j] = eligible[j+1]
            eligible[j+1] = temp

print("Ranked Scholarship List:")
for rank in range(len(eligible)):
    print(rank + 1, "-", eligible[rank][2], "| Score:", round(eligible[rank][0],
2))
```